EE3A Rivo Computer Organization HW4

1. for a 5-stage pipelined datapath

```
add $s1, $s2, $s1
lw $s3, 4($s1)
lw $s4, 0($s1)
sub $s3, $s4, $s3
sw $s3, 0($s1)
addi $s6, $s4, -1
sw $s6, 4($s5)
```

(a)

add $s1, $s2, $s1	IF	ID	EX	MEM	WB				
NOP									
NOP									
lw $s3, 4($s1)				IF	ID	EX	MEM	WB	
lw $s4, 0($s1)					IF	ID	EX	MEM	WB
NOP									
NOP									
sub $s3, $s4, $s3								IF	ID
NOP									
NOP									
sw $s3, 0($s1)									
addi $s6, $s4, -1									
NOP									
NOP									
sw $s6, 4($s5)									

add $s1, $s2, $s1										
NOP										
NOP										
lw $s3, 4($s1)										
lw $s4, 0($s1)										
NOP										
NOP										
sub $s3, $s4, $s3	EX	MEM	WB							
NOP										
NOP										
sw $s3, 0($s1)		IF	ID	EX	MEM	WB				
addi $s6, $s4, -1			IF	ID	EX	MEM	WB			
NOP										
NOP										
sw $s6, 4($s5)						IF	ID	EX	MEM	WB

(b)Yes, it is helpful. By rearranging the assembly code, we are able to dump 4 nops with the following MIPS code sequence.

add $s1, $s2, $s1
NOP
NOP
lw $s3, 4($s1)
lw $s4, 0($s1)
addi $s6, $s4, -1
NOP
sub $s3, $s4, $s3
sw $s6, 4($s5)
NOP
sw $s3, 0($s1)

(c)

EX/MEM.RegisterRd , MEM/WB.RegisterRd, ID/EX.RegisterRs, ID/EX.RegisterRt

(d)

use signal ForwardA and ForwardB to decide whether to forward data from ALU or MEM

MUX1(output=ALU_in1, Readdata1(0), MEM/WB.RegRd(1), EX/MEM.RegRd(2), CS=ForwardA)

MUX2(output=ALU_in2, Readdata2(0), MEM/WB.RegRd(1), EX/MEM.RegRd(2), CS=ForwardB)

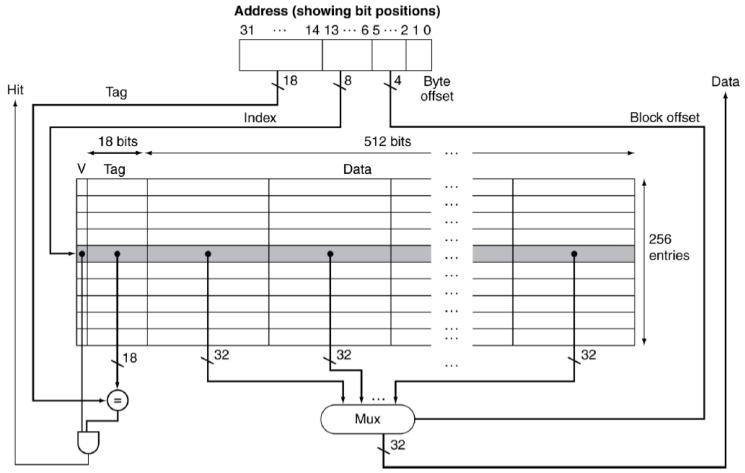
2. cache={valid,tags,data}
   cache size=blocks_number*(valid+tags+bits_per_block)
   index bits=$\log_2$(blocks_number)
   offset bits=block_offset+byte_offset=$\log_2$(words_per_block*bytes_per_word)
   tags+index+offset=32 bits(address)

(a) tag bits=32-$\log_2$(256)-$\log_2$(8*4)=19

(b) cache size=256*(1+19+8*32)=70656 bits
   words number=256*8=2048
   word-length = 32bits

(c) 2018= 00000000000000000000_00111111_00010 => index=00111111=63

(d)



3. set=#blocks/ways

(a) #set=128/8=16

(b) tag bits=32-$\log_2$(128/8)-$\log_2$(4*4)=24

#bits for block index=$\log_2$(#set)=4

(c) cache size= blocks_number*(valid+tags+bits_per_block)=128*(1+24+4*32)=19584 bits

(d)

